Research Project

DecoyFlix

RILEY KUIPER

KAYAN HO

CHAMARI ABESINGHE

PROJECT PLANNING

- Feature – Streamline Setup with Environment Validation & Dry-Run Mode
  1. Create environment pre-check
  2. Implement dependency verification
  3. Add LLM configuration detection
  4. Validate required files and paths
  5. Implement dry-run mode
- Feature – Improve Performance
  1. LLM Caching
  2. Parallelize plan generation
  3. Parallelize code generation
  4. Profiling
- Feature – Add GUI
  1. Window
  2. Input
  3. Execute Button
  4. Results
  5. Progress Indicator

DEVELOPMENT PROGRESS

STREAMLINE SETUP with Environment Walidation & Dry – Run Mode

- **What I changed**
- Added **environment pre-check script (`env_check.py`)**
- Implemented:
  - Python version & disk space validation
  - Dependency verification (e.g., torch)
  - LLM configuration detection
  - File & path validation
- Integrated checks into **`RepGen.sh`**
- Added **dry-run mode (`--dry-run`)**:
  - Runs minimal test (1 bug)
  - Skips heavy steps (baseline, ablations)
  - Bypasses user prompts

- **How I Tested**
- Ran full pipeline with valid setup
- Ran **dry-run mode** with missing dependencies
- Verified:
  - Errors block execution (normal mode)
  - Warnings allow execution (dry-run)
- Tested edge cases (low disk, missing LLM, missing files)

DEVELOPMENT PROGRESS

STREAMLINE SETUP with Environment Walidation & Dry –Run Mode

- **What I Learned**
- RepGen depends heavily on:
  - External tools (Ollama, OpenAI API)
  - Large model setups
- Initial setup is **complex and error-prone**
- Script workflow is **sequential and modular**

- **Progress vs Milestone Goals**
- ✅ Achieved: Improved usability & setup reliability
- ⚠️ Challenge: Environment setup (disk space, dependencies)
- ✅ Added extra value: Dry-run mode for testing

DEVELOPMENT PROGRESS
IMPROVE PERFORMANCE

- Implemented
  - File based cache for identical LLM queries (**`llm_cache.py`**)
  - Updated multiprocessing in **`parallel_processor.py`** for Plan and Code Generation
  - Updated **`tool.py`** to use cache, parallel execution, and profiling
- Testing
  - Used **`tool.py`** to confirm cache logic and integration

```python
_cache_instance: Optional[LLMCache] = None


def get_cache(cache_dir: str = ".llm_cache") -> LLMCache:
    """
    Get or create the global LLM cache instance.

    Args:
        cache_dir: Directory to store cache files

    Returns:
        LLMCache instance
    """
    global _cache_instance
    if _cache_instance is None:
        _cache_instance = LLMCache(cache_dir)
    return _cache_instance
```

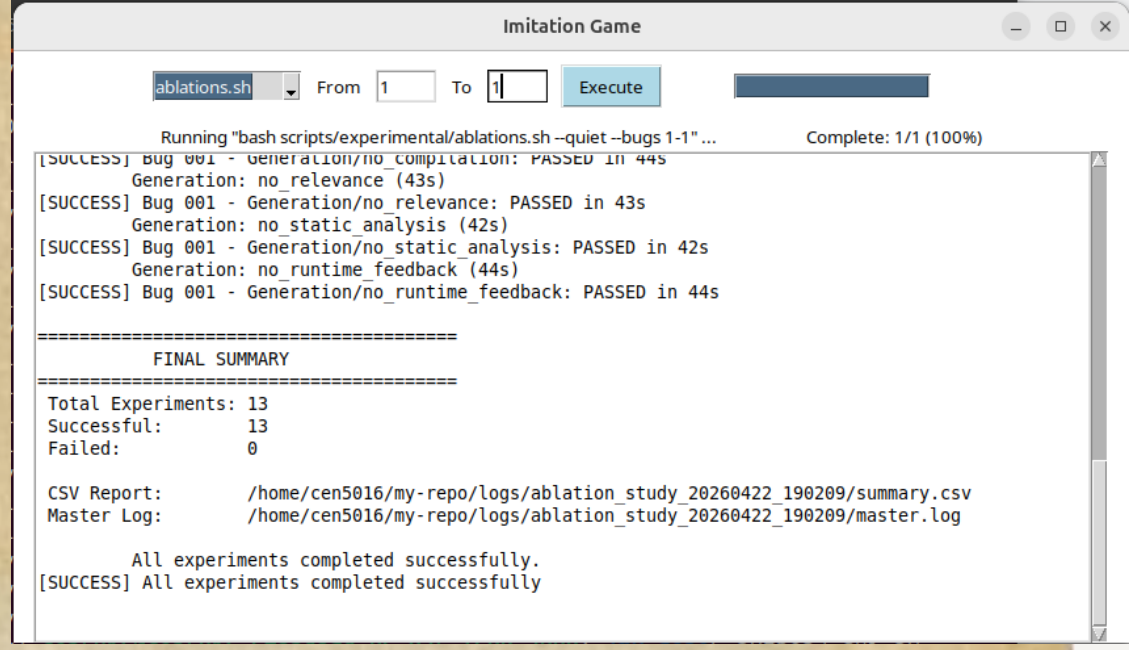
- What was learned
  - Pipeline runs sequential context loops that can be parallelized
  - Ollama subprocess calls are main bottleneck

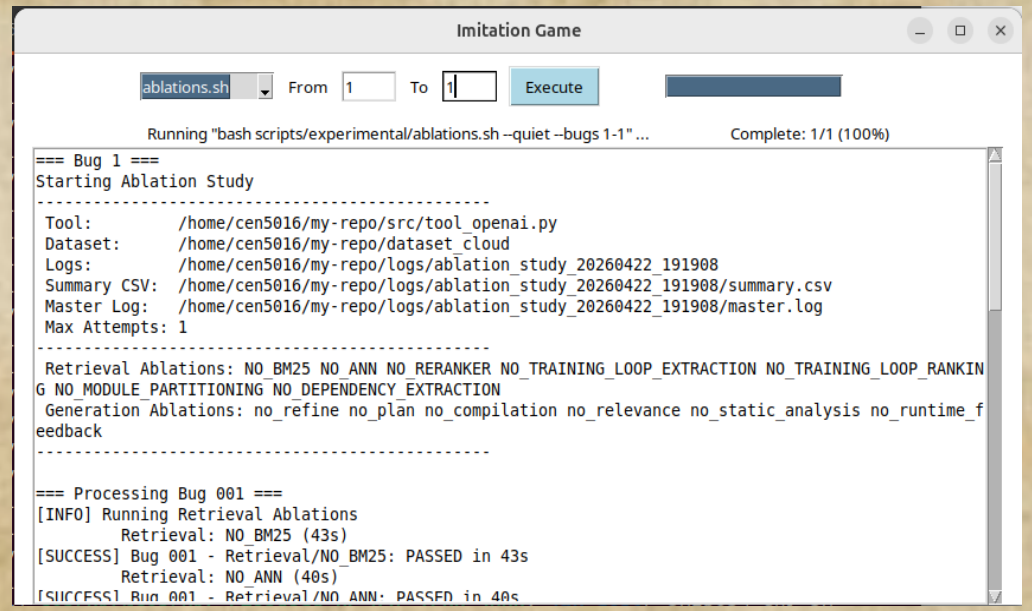
```python
class ParallelProcessor:
    """
    Manages parallel execution of tasks using multiprocessing.
    """

    def __init__(self, num_workers: Optional[int] = None):
        """
        Initialize the parallel processor.

        Args:
            num_workers: Number of worker processes (default: CPU count)
        """
        self.num_workers = num_workers or cpu_count()
        logger.info(f"Parallel processor initialized with {self.num_workers} workers")
```

DEVELOPMENT PROGRESS
GRAPHICAL USER INTERFACE

The "Add GUI" feature has been fully implemented

- Launches a GUI window that can be closed or minimized.
- A script can be selected via a dropdown, and a range of bugs can be entered.
- An execute button is present to run the selected script for the given range of bugs.
- Results are printed to a scrollable text area after each bug.
- A progress indicator is present that updates after the completion of every bug.





- Testing was manual. I ran the GUI, and sometimes printed to the terminal, in order to diagnose problems.
- Since the GUI is mostly external to the core program and simply runs each script, I have only learned how to operate the core program, what the scripts do, and their respective outputs.
  - However, I did learn about Tkinter, threads, and how to run bash scripts from Python.
- The "Add GUI" feature has been fully implemented, as specified for the Sprint 1 milestone goals.

LIVE DEMONSTRATION

REPGEN

WE WILL NOW DEMONSTRATE OUR CURRENT VERSION OF REPGEN

PROCESS OVERVIEW

1. Our team meets every Wednesday at 1:30 PM at the library.
2. GitHub branches created per user story; never pushed directly to main.
3. When a team member makes a pull request into main, another team member will review it.
4. Points 2 & 3 are enforced via branch protection rules.

Successes

- Due to branch protection rules, we have been able to avoid accidentally pushing directly to main.
- No merge conflicts
- We have rarely ever missed meetings after switching to Wednesday only, in person.

Difficulties

- As before, we don't commit often enough. However, we have more promptly reviewed each other's pull requests compared to last time.

OUR FUTURE PLANNING

We met all planned development goals and can submit our pull request to the original repo as planned.

We have completed the 3 features we planned to implement this sprint. We do not need to rescope.

We do not need to re-evaluate our process. We have learned from our previous mistakes and have mostly prevented potential issues. We still need to work on committing more frequently, but that will simply take practice.

WHAT WE DID WELL

- Met regularly
- Completed all tasks planned for Sprint 1
- Good branch management
- No pushes directly to main

NEEDS IMPROVEMENT

- Too much left until the last minute
- Too slow to commit changes

THANK YOU

- DECOYFLIX -